ТОО ”Колледж Хекслет”

Лабораторная работа №4

Разделение маршрутов с помощью Express Router

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_2. Разделение маршрутов с помощью Express Router.

Цель: Научиться использовать express.Router(), выносить маршруты в отдельные модули, экспортировать и импортировать Router, подключать маршрутизаторы через app.use() и организовывать структуру серверного проекта по ресурсам.

Условие: RO4_2/lesson4/ЛР-4.pdf

Требования задания

• содержать каталог routes;

• содержать минимум три отдельных Router;

• использовать express.Router();

• экспортировать каждый Router через export default;

• импортировать routers в app.js;

• подключать их через app.use();

• использовать отдельный URL-префикс для каждого ресурса;

• сохранять существующие URL API после рефакторинга;

• не содержать маршруты товаров, категорий и заказов непосредственно в app.js;

• запускать только один HTTP-сервер через app.listen().

Ход выполнения

Маршруты товаров, категорий и заказов вынесены в три отдельных Router. Внутри Router используются локальные пути / и /:id; app.use() добавляет ресурсный префикс. Например, префикс /api/products и путь /:id образуют /api/products/:id. Сервер создаётся один раз в app.js.

Проект: RO4_2/lesson4

Запуск из каталога проекта: npm install, затем npm start. Адрес сервера: http://localhost:3000.

app.js

app.js

```
import express from 'express';
import { pathToFileURL, fileURLToPath } from 'node:url';
import productRoutes from './routes/productRoutes.js';
import categoryRoutes from './routes/categoryRoutes.js';
import orderRoutes from './routes/orderRoutes.js';
import { notFound } from './middleware/notFound.js';
import { errorHandler } from './middleware/errorHandler.js';
const app = express();
app.use(express.json());
app.get('/', (req, res) => res.send('Internet Shop API'));
app.get('/about', (req, res) => res.send('Учебный REST API магазина компьютерной
техники'));
app.get('/api', (req, res) => res.json({ name: 'Internet Shop API', version: '1.0',
resources: ['/api/products', '/api/categories', '/api/orders'] }));
app.get('/api/status', (req, res) => res.json({ status: 'ok', message: 'Server is
running', requestTime: req.requestTime }));
app.get('/api/shop', (req, res) => res.json({ name: 'Kirill Tech Store', description:
'Учебный интернет-магазин', currency: 'KZT' }));
app.use('/api/products', productRoutes);
app.use('/api/categories', categoryRoutes);
app.use('/api/orders', orderRoutes);
app.use(notFound);
app.use(errorHandler);
export default app;

if (process.argv[1] && import.meta.url === pathToFileURL(process.argv[1]).href) {
    const PORT = Number(process.env.PORT || 3000);
    app.listen(PORT, () => console.log(`Server: http://localhost:${PORT}`));
}
```

Листинг 1. app.js

routes/productRoutes.js

routes/productRoutes.js

```
import { AppError } from '../errors/AppError.js';
import { Router } from 'express';

import { validateId } from '../middleware/validateId.js';
let products = [
    {
        "id": 1,
        "name": "Keyboard",
        "price": 15000,
        "category": "Accessories",
        "inStock": true
    },
    {
        "id": 2,
        "name": "Mouse",
        "price": 8000,
        "category": "Accessories",
        "inStock": true
    },
    {
        "id": 3,
        "name": "Monitor",
        "price": 85000,
        "category": "Monitors",
        "inStock": false
    },
    {
        "id": 4,
        "name": "Laptop",
        "price": 420000,
        "category": "Computers",
        "inStock": true
    },
    {
        "id": 5,
```

Листинг 2. routes/productRoutes.js (часть 1)

routes/productRoutes.js

```
        "name": "Webcam",
        "price": 22000,
        "category": "Accessories",
        "inStock": false
    }
];
let nextId = 6;

function getAllProducts(query = {}) {
    return products.filter(product =>
        (query.category === undefined || product.category === query.category) &&
        (query.inStock === undefined || product.inStock === (query.inStock === 'true'))
&&
        (query.minPrice === undefined || product.price >= Number(query.minPrice)) &&
        (query.maxPrice === undefined || product.price <= Number(query.maxPrice))
    );
}
function findProductById(id) {
    return products.find(product => product.id === id);
}
function addProduct(data) {
    const product = { ...data, id: nextId++ };
    products.push(product);
    return product;
}
function updateProductById(id, changes) {
    const product = findProductById(id);
    if (!product) return null;
    Object.assign(product, changes);
    return product;
}
function deleteProductById(id) {
    const index = products.findIndex(product => product.id === id);
    if (index === -1) return null;
    return products.splice(index, 1)[0];
```

Листинг 3. routes/productRoutes.js (часть 2)

routes/productRoutes.js

```
}



async function getProducts(req, res, next) {
    try { res.json(await getAllProducts(req.query)); }
    catch (error) { next(error); }
}
async function getProductById(req, res, next) {
    try {
        const product = await findProductById(Number(req.params.id));
        if (!product) return next(new AppError('Product not found', 404));
        res.json(product);
    } catch (error) { next(error); }
}
async function createProduct(req, res, next) {
    try { res.status(201).json(await addProduct(req.body)); }
    catch (error) { next(error); }
}
async function updateProduct(req, res, next) {
    try {
        const product = await updateProductById(Number(req.params.id), req.body);
        if (!product) return next(new AppError('Product not found', 404));
        res.json(product);
    } catch (error) { next(error); }
}
async function deleteProduct(req, res, next) {
    try {
        const product = await deleteProductById(Number(req.params.id));
        if (!product) return next(new AppError('Product not found', 404));
        res.json(product);
    } catch (error) { next(error); }
}
```

Листинг 4. routes/productRoutes.js (часть 3)

routes/productRoutes.js

```
function productSettings(req, res) {
    res.json({ id: Number(req.params.id), notify: req.query.notify === 'true', changes:
req.body });
}

const router = Router();
router.param('id', validateId);
router.get('/', getProducts);
router.get('/:id', getProductById);
router.post('/', createProduct);
router.patch('/:id', updateProduct);
router.delete('/:id', deleteProduct);
router.patch('/:id/settings', productSettings);
export default router;
```

Листинг 5. routes/productRoutes.js (часть 4)

routes/categoryRoutes.js

routes/categoryRoutes.js

```
import { Router } from 'express';
const router = Router();
const categories = ['Computers', 'Accessories', 'Monitors'];
router.get('/', (req, res) => res.json(categories));
router.post('/', (req, res) => res.status(201).json({ message: 'POST route: category
creation' }));
export default router;
```

Листинг 6. routes/categoryRoutes.js

routes/orderRoutes.js

routes/orderRoutes.js

```
import { Router } from 'express';
const router = Router();
const orders = [];
let nextId = 1;
router.get('/', (req, res) => res.json(orders));
router.get('/:id', (req, res) => {
    const order = orders.find(item => item.id === Number(req.params.id));
    if (!order) return res.status(404).json({ message: 'Order not found' });
    res.json(order);
});
router.post('/', (req, res) => {
    const order = { status: 'new', ...req.body, id: nextId++ };
    orders.push(order);
    res.status(201).json(order);
});
router.patch('/:id', (req, res) => {
    const order = orders.find(item => item.id === Number(req.params.id));
    if (!order) return res.status(404).json({ message: 'Order not found' });
    if (typeof req.body?.status === 'string') order.status = req.body.status;
    res.json(order);
});
export default router;
```

Листинг 7. routes/orderRoutes.js

Проверка результата

Проверено: 05.11.2026 13:44 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Обязательные endpoints

• HTTP-статусы и содержимое ответов

• Полный CRUD-цикл

• Совместная фильтрация

• Уникальные id после удаления

HTTP-запросы

GET / → 200

GET /about → 200

GET /api/status → 200

GET /api/shop → 200

GET /api/products → 200

GET /api/categories → 200

GET /api → 200

GET /api/orders → 200

GET /api/products/3 → 200

GET /api/products/999 → 404

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?inStock=false → 200

POST /api/products → 201

GET /api/products/6 → 200

PATCH /api/products/6 → 200

PATCH /api/products/4/settings?notify=true → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

PATCH /api/products/6 → 404

DELETE /api/products/6 → 404

POST /api/products → 201

GET /api/unknown → 404

POST /api/orders → 201

GET /api/orders/1 → 200

PATCH /api/orders/1 → 200

GET /api/orders/999 → 404

Примеры ответов

POST /api/products → 201


```
{
  "name": "SSD 2TB",
  "price": 65000,
  "category": "Components",
  "inStock": true,
  "id": 6
}
```


PATCH /api/products/6 → 200


```
{
  "name": "SSD 2TB",
  "price": 42000,
  "category": "Components",
  "inStock": false,
  "id": 6
}
```


DELETE /api/products/6 → 200


```
{
  "name": "SSD 2TB",
  "price": 42000,
  "category": "Components",
  "inStock": false,
  "id": 6
}
```


Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.